

Posto: Probabilistic Safety Monitoring — Full User Guide

Introduction — What Posto Does and How It Works

Posto helps you answer: “Given uncertainty, does my system stay within safety limits?”

It does this in three layers:

1. **Trajectory simulation** — Starting from an **initial set** (a box for each state), Posto evolves the system forward for `T` steps using either:
 - **Equation mode**: update rules written as equations; or
 - **ANN mode**: a neural network that predicts next state.
2. **Uncertain logging** — Instead of logging a single point, Posto records each observed state as an **interval** `[lo, hi]` (width controlled by `--dtlog`). You may also **probabilistically downsample** logs with `--prob` to emulate partial observability.
3. **Safety evaluation** — Posto reads **all constraints** from your **model JSON** and decides if the trajectory (or set of trajectories) ever violates any constraint. Internally, Posto can leverage **Jeffreys Bayes Factor (JBF)** to adaptively determine when enough evidence exists to declare **SAFE** or **UNSAFE** with confidence.

Key notions at a glance

- **State**: vector `[x, y, z, ...]` at a time step.
- **Equation update**: computes next state from current state and constants/noise.
- **Uncertainty**: injected as small random perturbations (e.g., `ep_x ∈ [-0.005, 0.005]`) and as **log-interval width** (`--dtlog`).
- **Constraint**: a rule like `x ≥ 10`, `y ≤ -8`, etc. Posto enforces **all** constraints from the JSON (logical OR of violations).
- **Verdict**: *SAFE* if **no** constraint is violated; *UNSAFE* if **any** constraint is violated at any logged time.

Installation & Environment

```
# Python ≥ 3.9 recommended
pip install numpy scipy mpmath tqdm matplotlib
python -c "import numpy,scipy,mpmath,tqdm,mpl_toolkits;print('All dependencies installed successfully')"
```

Set the project root (so imports like `from lib.Equation import Equation` work):

```
export MNTR_BB_ROOT_DIR=/path/to/Posto/
```

Add the above line to `~/ .bashrc` or `~/ .zshrc` to persist.

Repo layout (typical):

```
Posto/
├─ posto.py           # CLI entry
├─ system.py          # Core orchestrator
├─ model.py           # (Optional) user-defined getNextState()
├─ lib/
│   ├─ Equation.py    # Equation engine
│   ├─ ANN.py         # Neural-net engine
│   ├─ GenLog.py       # Uncertain log builder
│   ├─ TrajSafety.py   # Safety classification utilities
│   ├─ JFBF.py         # Bayes factor logic (sampling/stop)
│   └─ visualize.py    # 2D/3D plotting
└─ models/            # Your model JSONs
```

Model JSON (Equation Mode) — Spec & Example

File: `models/example_system.json`

```
{
  "state_vars": ["x", "y", "z"],
  "constants": {
    "dt": 0.01,
    "alpha": 2.0,
    "beta": 0.5,
    "gamma": 1.0
  },
  "ranges": {
    "ep_x": [-0.005, 0.005],
    "ep_y": [-0.002, 0.002],
    "ep_z": [-0.003, 0.003]
  },
  "equations": {
    "x": "x + dt * (alpha * y - beta * x**3) + ep_x",
    "y": "y + dt * (beta * x**2 - gamma * z) + ep_y",
    "z": "z + dt * (x - y + gamma * z**2) + ep_z"
  },
  "constraints": [
    { "state": "x", "op": "ge", "const": 10.0 },
    { "state": "x", "op": "le", "const": -10.0 },
    { "state": "y", "op": "ge", "const": 8.0 },
    { "state": "y", "op": "le", "const": -8.0 },
    { "state": "z", "op": "ge", "const": 12.0 },
    { "state": "z", "op": "le", "const": -12.0 }
  ]
}
```

Section-by-section

- `state_vars` — ordered names of states. Their **order matters** for indexing downstream ($x \rightarrow 0$, $y \rightarrow 1$, $z \rightarrow 2$).
- `constants` — parameters available in equations (e.g., `dt`, `alpha`, `beta`, `gamma`).
- `ranges` — symbols you can use as random perturbations inside equations (e.g., `ep_x`). At each step, the engine samples a value uniformly from the given interval.
- `equations` — right-hand-side expressions to compute each next state; you can use state names, constants, and range symbols. Pythonic operators apply (`**` for power).
- `constraints` — list of unsafe conditions **automatically enforced** during safety checks. Operators: `lt`, `le`, `gt`, `ge`, `eq`, `ne`.

Tip: Keep noise magnitudes (`ranges`) consistent with your logging width (`--dtlog`); if `ranges` $\ll$ `dtlog`, the log uncertainty will dominate.

Command Reference

Generate Uncertain Logs

```
python posto.py generateLog --log=logs/run_example.lg --init="[ -0.5, 0.5 ], [ -1.0, 1.0 ], [ -2.0, 2.0 ]" --timestamp=200 --mode=equation --model_path=models/example_system.json --prob=50 --dtlog=0.01
```

Parameters

- `--init` — one interval **per state** in the same order as `state_vars`.
- `--timestamp` — number of steps. Larger values expand the chance of reaching unsafe regions.
- `--prob` — percent of steps recorded to the log (e.g., 30, 50, 100).
- `--dtlog` — interval attached to each logged scalar value.

Sample log (excerpt) — `logs/run_example.lg`:

```
t=0: [ -0.010, 0.010 ], [ -0.020, 0.020 ], [ -0.015, 0.015 ]
t=25: [ 0.234, 0.244 ], [ 0.120, 0.130 ], [ 0.007, 0.017 ]
t=100: [ 0.850, 0.860 ], [ -0.320, -0.310 ], [ 0.420, 0.430 ]
t=180: [ 1.100, 1.110 ], [ -0.640, -0.630 ], [ 0.890, 0.900 ]
```

Check Safety (Multi-constraint)

```
python posto.py checkSafety --log=logs/run_example.lg --mode=equation --model_path=models/example_system.json
```

- Reads **all** constraints from the JSON and flags a violation if **any** one fails.
- Expected OK output:

```
[INFO] Loaded constraints: 6
[SUCCESS] System is SAFE (no violations detected).
```

- Example violation output:

```
[WARN] Violation detected: state x >= 10.0 at t=176
[FAIL] System is UNSAFE.
```

In-Depth: Using `Model.py` for Custom Dynamics

Sometimes you want full Python control (algorithmic logic, conditionals, calls to third-party libs) instead of JSON equations. The `Model.py` path lets you **override** the system's next-state map at runtime.

Minimal custom model

Model.py

```
# Model.py
import random

def my_getNextState(state):
    """
    state: tuple/list of floats, e.g., (x, y)
    return: next-state tuple of same length
    """
    x, y = state
    # simple drift with tiny noise
    x = x + 0.1 + random.uniform(-0.01, 0.01)
    y = y - 0.05 + random.uniform(-0.01, 0.01)
    return (x, y)
```

Use it with System

```
# examples/run_with_model.py
from System import System
from Model import my_getNextState

# no mode/model_path: we'll inject our own step function
sys = System(log_path="logs/custom_with_model.py.log", mode=None, model_path=None)

# override the engine step
sys.getNextState = my_getNextState

# generate a log: init set must match state dimension (x, y)
init_box = [[0.0, 1.0], [0.0, 2.0]]
sys.generateLog(init_box, T=100, prob=50, dtlog=0.01)
```

Run

```
python examples/run_with_model.py.py
```

Custom model with 3 states (x, y, z) and parameters

```
# Model.py
import random
ALPHA, BETA, GAMMA, DT = 2.0, 0.5, 1.0, 0.01

def get_noise(lo, hi):
    return random.uniform(lo, hi)

def my_xyz_step(state):
    """A Python equivalent to the JSON equations"""
    x, y, z = state
    ep_x = get_noise(-0.005, 0.005)
    ep_y = get_noise(-0.002, 0.002)
    ep_z = get_noise(-0.003, 0.003)

    x_next = x + DT * (ALPHA * y - BETA * x**3) + ep_x
    y_next = y + DT * (BETA * x**2 - GAMMA * z) + ep_y
    z_next = z + DT * (x - y + GAMMA * z**2) + ep_z
    return (x_next, y_next, z_next)
```

Plug it into System

```
from System import System
from Model import my_xyz_step

sys = System(log_path="logs/custom_xyz.log", mode=None, model_path=None)
sys.getNextState = my_xyz_step

# 3D init set for (x,y,z)
init_xyz = [[-0.5, 0.5], [-1.0, 1.0], [-2.0, 2.0]]
sys.generateLog(init_xyz, T=200, prob=50, dtlog=0.01)
```

Note: When using `Model.py`, constraints are not embedded here. For safety checks you have two options:

1. Use a **JSON** model file that contains only the `constraints` block and pass `--model_path` to `checkSafety` so it can read constraints, or
2. Add a tiny wrapper that supplies constraints to the safety checker programmatically (if your codebase supports it).

The simplest path in your current setup is **(1)** — keep constraints in a JSON and use it for `checkSafety`.

5.3 Hybrid pattern: JSON constraints + Model.py dynamics

- **Step A:** Keep `models/constraints_only.json` containing just `state_vars` and `constraints` (constants/equations optional).
- **Step B:** Generate logs via your `Model.py` step function (as above).
- **Step C:** Run safety check pointing to the constraints JSON:

```
python posto.py checkSafety --log=logs/custom_xyz.lg --mode=equation --  
model_path=models/constraints_only.json
```

This lets you iterate fast on Python dynamics while centralizing constraints in one place.

5.4 Testing your Model.py step

```
def smoke_test():  
    s = (0.1, -0.2, 0.0)  
    ns = my_xyz_step(s)  
    assert len(ns) == 3 and all(isinstance(v, (int, float)) for v in ns)  
    print("Model.py step OK:", ns)  
  
if __name__ == "__main__":  
    smoke_test()
```

Worked End-to-End Example (JSON equations)

Prepare the model

- Save the **3-state** JSON shown earlier as `models/example_system.json`.

Generate log

```
python posto.py generateLog --log=logs/run_example.lg --init="[-0.5, 0.5 ], [  
-1.0, 1.0 ], [ -2.0, 2.0 ]" --timestamp=200 --mode=equation --  
model_path=models/example_system.json --prob=50 --dtlog=0.01
```

Safety check (uses all JSON constraints)

```
python posto.py checkSafety --log=logs/run_example.lg --mode=equation --  
model_path=models/example_system.json
```

Expected (safe case):

```
[INFO] Loaded constraints: 6  
[SUCCESS] System is SAFE (no violations detected).
```

Possible (unsafe case):

```
[WARN] violation detected: state y <= -8.0 at t=172  
[FAIL] System is UNSAFE.
```

Visualization & Tips

Quick plot of a log

```
from lib.Visualize import Visualize  
viz = Visualize(viz=True, msg=None)  
viz.vizLog("logs/run_example.lg", save=True) # saves images to your img dir
```

Tuning guidelines

- Increase `--timestamp` to stress the system longer.
- Increase `--prob` to capture more steps (denser logs).
- Match `ranges` noise and `--dtlog` so intervals are informative but not overly wide.
- Use narrower `--init` boxes to study local behavior; widen to explore robustness.

Common pitfalls

- Mismatch between `state_vars` length and `--init` intervals.
- Using variable names in equations not defined in `state_vars` or `constants`.
- Oversized `--dtlog` making constraints trivially “violated” by interval overlap.